MINI PROJECT

NAME: PREETAMSINGH DONUR

REGISTER NUMBER: 711525BCS122

BATCH: Computer Science and Engineering

MINI PROJECT TITLE: Cinema Booking system

COURSE: DATA STRUCTURES (B25CSI201)

GitHub Repository:

ABSTRACT:

- The system manages cinema ticket reservations dynamically based on Ticket IDs.
- It ensures booking records are kept in a sorted sequence upon entry.
- It implements Binary Search to achieve high-speed retrieval of customer data.

PROBLEM STATEMENT:

- Cinemas must rapidly search for customer bookings among thousands of daily transactions.
- Linear search methods cause massive delays during peak movie times.
- The system requires an efficient way to insert, sort, and search reservations using array data structures.

OBJECTIVES:

- To manage ticket reservations efficiently without data loss.
- To maintain a strictly sorted array for all incoming bookings.
- To search for existing patient or customer records rapidly using Binary Search.

DATA STRUCTURES USED:

Array / Array List: Used to store the cinema ticket records in memory.

ALGORITHMS USED:

- Insertion Logic: To shift elements and insert new bookings in sorted order.
- Binary Search: To find ticket records by repeatedly dividing the search interval in half.

PSEUDOCODE:

INSERT BOOKING: Insert(ticketId, name, movie):

- ☐ Check if the array is full.
- ☐ Start a loop from the last existing element backwards.
- ☐ If the current element's Ticket ID > new Ticket ID, shift the element one position to the right.
- ☐ Insert the new booking into the correctly sorted empty slot.

BINARY SEARCH: Search(targetId):

- ☐ Set low = 0 and high = count - 1.
- ☐ Loop while low <= high.
- ☐ Calculate mid = (low + high) / 2.
- ☐ If array[mid] == targetId, return the booking details.
- ☐ If array[mid] < targetId, set low = mid + 1.
- ☐ If array[mid] > targetId, set high = mid - 1.
- ☐ Return "Not Found" if the loop ends.

TIME AND SPACE COMPLEXITY:

- Insertion: O(n)
- Binary Search: O(\log n)
- Space Complexity: O(n)

JAVA PROGRAMM:

```
import java.util.Scanner;

class Booking {
    int ticketId;
    String customerName;
    String movieName;

    Booking(int ticketId, String customerName, String movieName) {
        this.ticketId = ticketId;
        this.customerName = customerName;
        this.movieName = movieName;
    }
}

public class CinemaBookingSystem {
    Booking[] reservations;
    int count;

    public CinemaBookingSystem(int capacity) {
        reservations = new Booking[capacity];
        count = 0;
    }

    // Inserts booking directly into its sorted position
    public void addBooking(int ticketId, String name, String movie) {
```

```
    if (count == reservations.length) {
        System.out.println("System full. Cannot accept more bookings.");
        return;
    }

    Booking newBooking = new Booking(ticketId, name, movie);
    int i = count - 1;

    // Shift elements to the right to make space for the new sorted ID
    while (i >= 0 && reservations[i].ticketId > ticketId) {
        reservations[i + 1] = reservations[i];
        i--;
    }

    reservations[i + 1] = newBooking;
    count++;
    System.out.println("✅ Booking added successfully for " + name);
}

// Binary Search Implementation
public void searchBooking(int targetId) {
    int low = 0;
    int high = count - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;
```

```
            if (reservations[mid].ticketId == targetId) {
                System.out.println("\n🎬 Booking Found!");
                System.out.println("Ticket ID: " + reservations[mid].ticketId);
                System.out.println("Customer: " +
reservations[mid].customerName);
                System.out.println("Movie: " + reservations[mid].movieName);
                return;
            }

            if (reservations[mid].ticketId < targetId) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
        System.out.println("\n❌ No booking found with Ticket ID: " +
targetId);
    }

    public void displayBookings() {
        if (count == 0) {
            System.out.println("No reservations currently in the system.");
            return;
        }
        System.out.println("\n--- Current Bookings (Sorted by ID) ---");
        for (int i = 0; i < count; i++) {
            System.out.println("ID: " + reservations[i].ticketId + " | Name: " +
reservations[i].customerName + " | Movie: " + reservations[i].movieName);
```

```
    }
  }

  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    CinemaBookingSystem system = new CinemaBookingSystem(100);
    int choice;

    do {
      System.out.println("\n1. Add Booking\n2. Search Booking (Binary
Search)\n3. Display All Bookings\n4. Exit");
      System.out.print("Enter choice: ");
      choice = sc.nextInt();

      switch (choice) {
        case 1:
          System.out.print("Enter Ticket ID (Number): ");
          int id = sc.nextInt();
          sc.nextLine(); // consume newline
          System.out.print("Enter Customer Name: ");
          String name = sc.nextLine();
          System.out.print("Enter Movie Name: ");
          String movie = sc.nextLine();
          system.addBooking(id, name, movie);
          break;
        case 2:
          System.out.print("Enter Ticket ID to search: ");
          int searchId = sc.nextInt();
```

```
                system.searchBooking(searchId);
                break;
            case 3:
                system.displayBookings();
                break;
            case 4:
                System.out.println("Exiting system. Have a great day! 🏠");
                break;
            default:
                System.out.println("Invalid choice. Try again.");
        }
    } while (choice != 4);

    sc.close();
  }
}
```

SAMPLE OUTPUT:

I. Adding First Booking

1. Add Booking

Enter Ticket ID (Number): 105

Enter Customer Name: Ravi

Enter Movie Name: Inception

✅ Booking added successfully for Ravi

II. Adding Second Booking (Testing Sorting - Smaller ID)

1. Add Booking

Enter Ticket ID (Number): 102

Enter Customer Name: Arun

Enter Movie Name: Interstellar

✅ Booking added successfully for Arun

II. Displaying Bookings (Verifying Sorted Order)

3. Display All Bookings

--- Current Bookings (Sorted by ID) ---

ID: 102 | Name: Arun | Movie: Interstellar

ID: 105 | Name: Ravi | Movie: Inception

III. Searching for an Existing Booking (Binary Search Success)

2. Search Booking (Binary Search)

Enter Ticket ID to search: 105

Booking Found!

Ticket ID: 105

Customer: Ravi

Movie: Inception

IV. Searching for a Non-Existent Booking (Binary Search Failure)

2. Search Booking (Binary Search)

Enter Ticket ID to search: 999

❌ No booking found with Ticket ID: 999

REFERENCES:

Introduction to Algorithms by Cormen Java Documentation (Scanner and Array Concepts) Data Structures and Algorithms textbooks GeeksforGeeks (Binary Search & Array Insertion concepts)